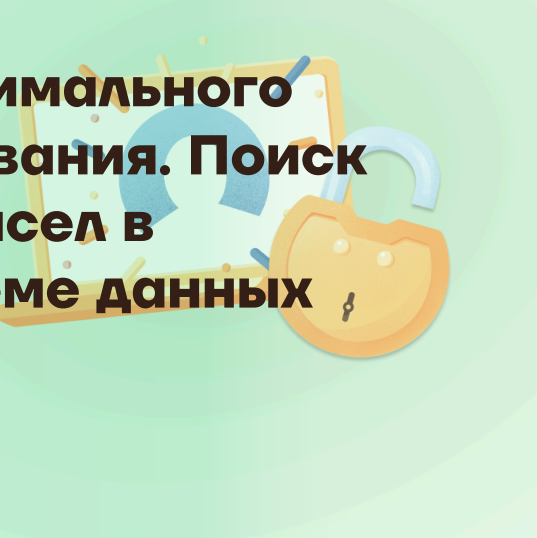
Умскул учебник стремится стать лучше! Если вы наткнулись на ошибку или неточность в нашем материале - просто сообщите нам, мы будем благодарны!

Оставить отзыв

← К СПИСКУ СТАТЕЙ

ИНФОРМАТИКА

Практика оптимального программирования. Поиск нескольких чисел в большом объеме данных



19.5.2022 5306

ЕГЭ ОГЭ

Еженедельные бесплатные вебинары от преподавателей УМСКУЛ

ЗАПИСАТЬСЯ

Поделитесь статьей в социальных сетях:

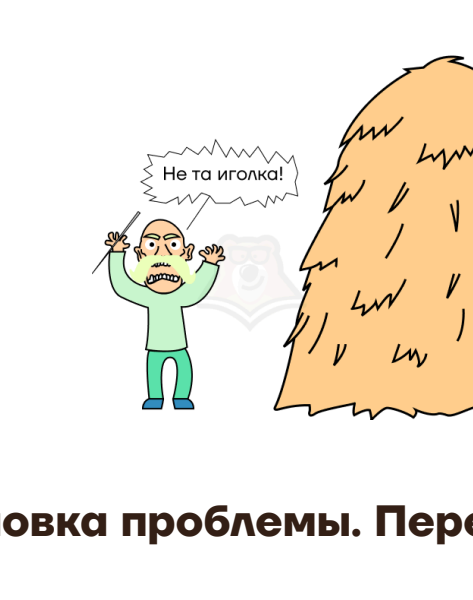
О чем эта статья:

Постановка проблемы. Переборное решение
Чтение данных из файла
Произведение множителей
Остатки от деления
Фактчек
Проверь себя

На этой странице вы узнаете

- Как научить программу читать по-настоящему?
- Цикл долой: как одной строкой изменить все данные списка?
- Почему вообще одно число делится на другое?

Трудно найти иголку в стоге сена? Труднее найти две. А еще труднее — найти две конкретные иголки, которые будут дополнять друг друга. Давайте поговорим, как можно решить такую задачу быстро и эффективно при помощи программирования.


Не та иголка!

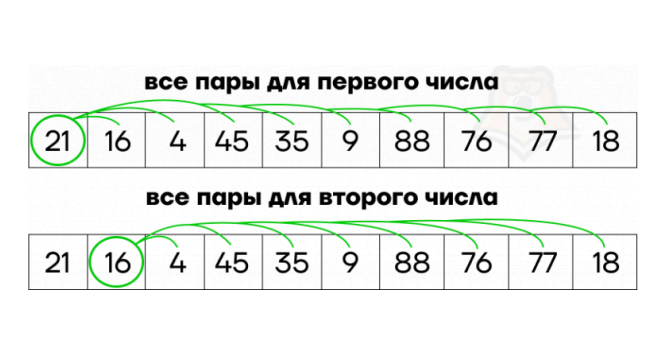
Постановка проблемы. Переборное решение

Анализ больших данных — частая проблема, с которой приходится сталкиваться программистам. И дело не в том, что это сложно. **Сложно сделать это быстро**.

В статье «Практика оптимального программирования. Поиск делителей числа» мы уже занимались поиском подходящих чисел на достаточно больших диапазонах. Чтобы программа работала быстрее, мы находили способы сократить перебор и не считать лишние данные. **Такой подход работает не всегда**.

Например, на таком условии задачи: из указанного массива чисел выбрать два таких, чтобы их произведение было кратным 14 и наибольшим, а также вывести на экран это произведение.

Самое простое решение — перебор. Для каждого числа мы проверим все возможные пары, найдем из них подходящие и выберем максимальные. Пары будем рассматривать для каждого числа со всеми последующими в массиве.



Что нам понадобится для реализации такой программы?

1. Сам массив с числами.
2. Два цикла **for**, причем второй будет вложен в первый. Внешний цикл будет перебирать сами числа от первого до предпоследнего, внутренний — все последующие числа (от следующего числа до самого последнего).
3. Проверка подходящей пары.
4. Заранее созданная переменная, в которую мы сохраним максимальное подходящее число.

```
numbers = [21, 16, 4, 45, 35, 9, 88, 76, 77, 18]

max_couple = 0

for first in range(len(numbers) - 1):
        for second in range(first + 1, len(numbers)):
                if ((numbers[first] * numbers[second]) % 14 == 0 and max_couple <
numbers[first] * numbers[second]):
                        max_couple = numbers[first] * numbers[second]

print(max_couple)

Вывод: 6776
```

Прекрасно, все работает, и даже достаточно быстро. **Но для десяти чисел можно было не писать программу, мы бы и вручную справились.** Будет ли работать это решение, если чисел будет 100? 1000? 10000? И как передать такой объем данных в программу?

Чтение данных из файла

Когда данных очень много, неудобно записывать их прямо в программе. Проще хранить их в отдельном файле, и в нем же редактировать.

Как научить программу читать по-настоящему?

Отдельные текстовые данные можно хранить в отдельном текстовом файле формата .txt. Программа сможет их прочитать и использовать в дальнейшем.

Для открытия файлов в Python используется следующая запись:

<переменная> = open(“<путь к файлу>”, “<режим доступа>”)

Разберем элементы этой записи:

- Сам файл будет записан в указанную переменную, что позволит нам работать с ним как, собственно, с переменной.
- Команда **open** откроет файл по указанному пути.
- Если текстовый файл лежит в одной папке с файлом вашей программы, то в качестве пути файла можно указать только его имя — программа найдет его в своей же папке.

Если же файл находится в другом месте, необходимо прописывать полный путь

- Режим доступа — дополнительный параметр. Файл можно открыть так, чтобы его можно было читать, но нельзя в него ничего записывать. Или наоборот — открыть файл на запись, но тогда он будет переписан, а прочитать его мы не сможем.
  - режим “r” откроет файл только на чтение;
  - режим “w” — только на запись, причем этот режим полностью очистит документ;
  - режим “a” позволит дописывать информацию к имеющейся.

Если не указать этот параметр совсем, файл будет открыт по умолчанию только на чтение.

Чтобы считывать данные из файла, есть несколько способов:

- команда **readline** будет считывать по одной строке текстового файла;
- **readlines** создаст массив, в котором будут храниться сразу все строки файла;
- как строку или массив, файл можно перебирать с помощью цикла for.

Важно запомнить — читаются данные из файла именно в строковом формате. Если нужно преобразовать их к числам, делаем это с помощью команды int.

Если у нас есть текстовый документ text.txt, в котором прописаны необходимые числа — каждое в новой строке — у нас есть несколько способов создать массив, содержащий все эти числа. Для удобства, пусть в первой строке файла будет количество чисел для обработки.

Цикл долой: как одной строкой изменить все данные списка?

В последнем примере мы использовали функцию **map**. Она занимается тем, что применяет определенную команду ко всем элементам переданного ей массива. В данном случае мы создали массив со всеми числами файла командой **file.readlines().** Так как из файла считывается строковой тип данных, функцией map мы применили команду **int** сразу ко всем считанным числам. Но сам map список не создаст, потому мы ему поможем командой **list**.

В итоге результат один — в списке numbers будут находиться все числа из файла в количестве count штук.

Допишем эту часть в наш код и попробуем запустить:

```
file = open("text.txt")
count = int(file.readline())
numbers = list(map(int, file.readlines()))


max_couple = 0

for first in range(len(numbers) - 1):
        for second in range(first + 1, len(numbers)):
                if ((numbers[first] * numbers[second]) % 14 == 0 and max_couple <
numbers[first] * numbers[second]):
                        max_couple = numbers[first] * numbers[second]

print(max_couple)
```

Теперь засечем время, сколько будет работать программа для поиска пар чисел при разном количестве чисел в файле:

- 100 чисел — 0.1 секунда;
- 1000 чисел — 0.2 секунды;
- 10000 чисел — 6.5 секунды.

Давайте рассмотрим, как полученные знания могут помочь при выполнении **27 номера ЕГЭ**.

Задание. *Дана последовательность N целых положительных чисел. Рассматриваются все пары элементов последовательности. Из указанного массива чисел выбрать два таких, чтобы их произведение было кратным 15 и наибольшим. Вывести на экран это произведение.*

Решение.
По аналогии напишем код, о котором мы говорили выше.

```
file = open(«text.txt»)
count = int(file.readline())
numbers = list(map(int, file.readlines()))

max_couple = 0

for first in range(len(numbers) — 1):
        for second in range(first + 1, len(numbers)):
                if ((numbers[first] * numbers[second]) % 15 == 0 and max_couple <
numbers[first] * numbers[second]):
                        max_couple = numbers[first] * numbers[second]
print(max_couple)
```

В данном случае мы рассмотрели переборное решение. Как правило, оно подходит для небольших файлов, с небольшим количеством чисел.

Станет хуже, если мы начнем искать тройки чисел. Для этого понадобится немного изменить программу — добавить третий внутренний цикл и учесть третье число в проверке произведения:

```
file = open("text.txt")
count = int(file.readline())
numbers = list(map(int, file.readlines()))

max_couple = 0

for first in range(len(numbers) - 2):
        for second in range(first + 1, len(numbers) - 1):
                for third in range(second + 1, len(numbers)):
                        if ((numbers[first] * numbers[second] * numbers[third]) % 14
== 0 and max_couple < numbers[first] * numbers[second] * numbers[third]):
                                max_couple = numbers[first] * numbers[second] *
numbers[third]

print(max_couple)
```

И снова засекаем время:

- 100 чисел — 0.2 секунды;
- 1000 чисел — 45.5 секунды;
- 10000 чисел — за 8 часов программа так и не выполнилась.

Это явно не самый успешный результат. От перебора надо избавляться и придумывать что-то хитрее.

Но для этого нам потребуется вспомнить некоторые знания математики. Начнем с произведения множителей.

Произведение множителей

Почему вообще одно число делится на другое?

Все числа можно разложить на произведение простых множителей. Например:

4 = 2 * 2
12 = 2 * 2 * 3
22 = 2 * 11

И если все множители одного числа есть во втором, то второе число можно разделить на первое. Например, среди множителей 12 есть две двойки, а у 4 все множители — двойки. Значит, 12 можно разделить на 4.

По условию задачи нам нужно произведение, которое кратно 14. Число 14 можно разложить как 2 * 7. Значит, в искомом произведении среди множителей должны найтись 2 и 7. Например:

- Если умножить 4 на 21, в разложении получится 4 * 21 = 2 * 2 * 3 * 7. Среди множителей есть 2 и 7, тогда это произведение поделится на 14.
- Если умножить 14 на 17, мы получим 14 * 17 = 2 * 7 * 17. Так как в произведении уже есть 14, оно уже будет кратно 14, какое бы второе число мы ни подставили.
- Если умножить 15 на 18, получим 15 * 18 = 3 * 5 * 2 * 3 * 3. Семерки среди множителей нет, значит, это произведение не будет кратно ни 7, ни 14.

Значит, чтобы получить произведение, кратное 14, надо чтобы один из множителей был кратен 14, либо одно из чисел кратно 2, а второе — 7. Отсортируем заранее все числа из файла по группам: кратные 14, кратные 7, кратные 2 и не кратные ничему из этого. Тогда найти необходимое произведение не составит труда.

Оптимальная программа будет состоять из следующих элементов:

- массивов, в которые мы будем распределять числа;
- дополнительного условия распределения чисел в цикле чтения данных из файла — в зависимости от кратности числа будем заносить его в соответствующую группу;
- подсчет итогового ответа.

Как мы уже выяснили раньше, подходящей парой будет та, в которой одно число будет кратно 14, а второе — вообще любое, в том числе другое кратное 14. Либо нужна пара, в которой одно число кратно 2, а второе — 7. Так как нам нужно наибольшее произведение, будем брать наибольшие множители. Нам поможет команда max и сортировка по убыванию — наибольшие элементы будут в самом начале массива, их будет удобно брать по индексам.

Далее в этой статье будем работать вот с этим файлом: text.txt.

```
file = open("text.txt")
count = int(file.readline())

kr14 = []
kr7 = []
kr2 = []
nekr = []

for number in file:
        if int(number) % 14 == 0:
                kr14.append(int(number))
        elif int(number) % 7 == 0:
                kr7.append(int(number))
        elif int(number) % 2 == 0:
                kr2.append(int(number))
        else:
                nekr.append(int(number))

kr14.sort(reverse = True)
kr7.sort(reverse = True)
kr2.sort(reverse = True)
nekr.sort(reverse = True)

ans1 = kr14[0] * max(kr14[1], kr2[0], kr7[0], nekr[0])
ans2 = kr2[0] * kr7[0]

print(max(ans1, ans2))

Вывод: 99890000
```

Выполнение программы не заняло много времени.

Давайте рассмотрим, как оптимальное программирование может помочь при выполнении **27 номера ЕГЭ**. Посмотрим пример того же номера, о котором мы говорили выше.

Задание. *Дана последовательность N целых положительных чисел. Рассматриваются все пары элементов последовательности. Из указанного массива чисел выбрать два таких, чтобы их произведение было кратным 15 и наибольшим. Вывести на экран это произведение.*

Решение. Чтобы получить произведение, кратное 15, надо чтобы один из множителей был кратен 15, либо одно из чисел кратно 3, а второе – 5. Для этого у нас будут массивы!

На самом деле код аналогичен тому, что мы рассмотрели, но тут важно понять, что число 15 или 14 может меняться в зависимости от условия, поэтому каждый раз необходимо подбирать делители чисел.

```
file = open(«text.txt»)
count = int(file.readline())

kr15 = []
kr5 = []
kr3 = []
nekr = []

for number in file:
        if int(number) % 15 == 0:
                kr15.append(int(number))
        elif int(number) % 5 == 0:
                kr5.append(int(number))
        elif int(number) % 3== 0:
                kr3.append(int(number))
else:
                nekr.append(int(number))

kr15.sort(reverse = True)
kr5.sort(reverse = True)
kr3.sort(reverse = True)
nekr.sort(reverse = True)

ans1 = kr15[0] * max(kr15[1], kr5[0], kr3[0], nekr[0])
ans2 = kr5[0] * kr3[0]

print(max(ans1, ans2))
```

Умея читать большое количество данных из файла, а также умея грамотно распределять числа, давайте решим еще одну проблему — **поиск не произведения, а суммы**.

Остатки от деления

Теперь мы хотим найти пару чисел, **сумма которой будет кратна 17**. Как проверить кратность суммы?

Обратимся к математике.

Кратное 17 число можно представить как 17n, где n — натуральное число (1, 2, 3 и так далее).

Если число не кратно 17, то его можно представить как 17n + m, где m будет остатком от деления числа на 17.

Попробуем рассмотреть различные варианты суммы двух чисел:

- если сложить два числа, кратных 17, никаких проблем не будет — $17n_1 + 17n_2 = 17(n_1 + n_2)$, среди множителей сразу появляется 17, значит, сумма будет кратна 17;
- если числа будут с какими-то остатками от деления, то понадобится дополнительное условие: $17n_1 + m_1 + 17n_2 + m_2 = 17 * (n_1 + n_2) + m_1 + m_2$. Надо вынести 17 в общий множитель, чтобы все выражение было кратно 17.
  m_1 и m_2 не могут быть кратны 17 по отдельности, так как это остатки от деления. **Значит, их сумма должна быть кратна 17**, то есть $m_1 + m_2 = 17n_3$. Только тогда мы сможем вынести 17 общим множителем.

Нам снова придется сортировать числа, **но не по кратности, а по остаткам от деления**. Остатком от деления на 17 может быть любое число от 0 до 16. Здесь мы можем создать **двумерный список**, про который мы говорили в статье «Работа с массивами в Python». Нам нужно 17 внутренних списков, которые будут иметь индексы от 0 до 16. В каждый мы сможем заносить числа по остаткам от деления — чтобы остаток был равен индексу.

В коде нам понадобятся:

- Заранее созданный двумерный список с 17 пустыми списками внутри.
- Сортировка чисел файла по внутренним спискам. Какой остаток от деления на 17, такой и индекс внутреннего списка;
- Сортировка внутренних списков для удобства.
- Список, в который мы будем складывать потенциальные ответы.
- Потенциальные ответы — сумма наибольших чисел внутренних массивов с индексами i и 17 — i .В сумме они должны дать ровно 17 — это и обеспечит нам необходимую сумму. Перебирать i будет достаточно до 8. При i = 8 мы проверим индексы 8 и 9, а при i = 9 — 9 и 8, то есть начнем повторяться.
- Для i = 0, чисел, кратных 17, проверка должна быть отдельной, так как мы будем брать два таких наибольших числа.
- Перед подсчетом любой суммы — поверка:
  - есть числа с необходимыми остатками от деления — считаем;
  - нет таких чисел — считать нечего.
- Ответ — наибольший из потенциальных.

Следующий код полностью выполнит все необходимые требования, и работать будет очень быстро:

```
file = open("text.txt")
count = int(file.readline())

kr = []
for i in range(17):
        kr.append([])

for number in file:
        number = int(number)
        kr[number % 17].append(number)

for i in range(17):
        kr[i].sort(reverse = True)

ans = []

if len(kr[0]) >= 2:
        ans.append(kr[0][0] + kr[0][1])

for i in range(1, 9):
        if len(kr[i]) > 0 and len(kr[17 - i]) > 0:
                n = kr[i][0] + kr[17 - i][0]
                ans.append(n)

print(max(ans))

Вывод: 19992
```

Мы рассмотрели важный аспект оптимального программирования. Этот подход включает в себя постоянное обучение и развитие, рациональный выбор инструментов, эффективную организацию кода. Оптимальное программирование – непрерывный процесс, и каждый из этих аспектов может быть улучшен и адаптирован под конкретные ситуации!
Если вы хотите узнать больше о Python, рекомендуем ознакомиться со статьей «Основы программирования на языке Python. Часть 1».

Фактчек

- Для открытия файла в программе используется команда **open**, а для чтения данных из него — команды **readline** (читает одну строку) и **readlines** (считает сразу все).
- **Произведение чисел** будет кратно n, если среди множителей этого произведения можно найти такие, произведение которых будет равно n.
- **Сумма чисел** будет кратна n, если сумма их остатков от деления будет кратна n.

Проверь себя

Задание 1.
В каких ситуациях переборное решение не будет работать?

1. Во всех. Это решение бесполезно и не имеет права на существование.
2. В определенных, это всегда зависит от условия задачи.
3. При очень большом количестве перебираемых данных.
4. Ни в каких. Это хорошее решение, которое можно использовать в любой ситуации.

Задание 2.
В файле text.txt найдите наибольшее произведение двух чисел, которое будет кратно 21.

1. 99880011
2. 99878121
3. 99922211
4. Такого произведения нет

Задание 3.
В файле text.txt найдите наибольшую сумму двух чисел, которая будет кратна 11.

1. 11110
2. 19987
3. 19998
4. Такой суммы нет

Ответы: 1. — 3; **2.** — 1; **3.** — 3.

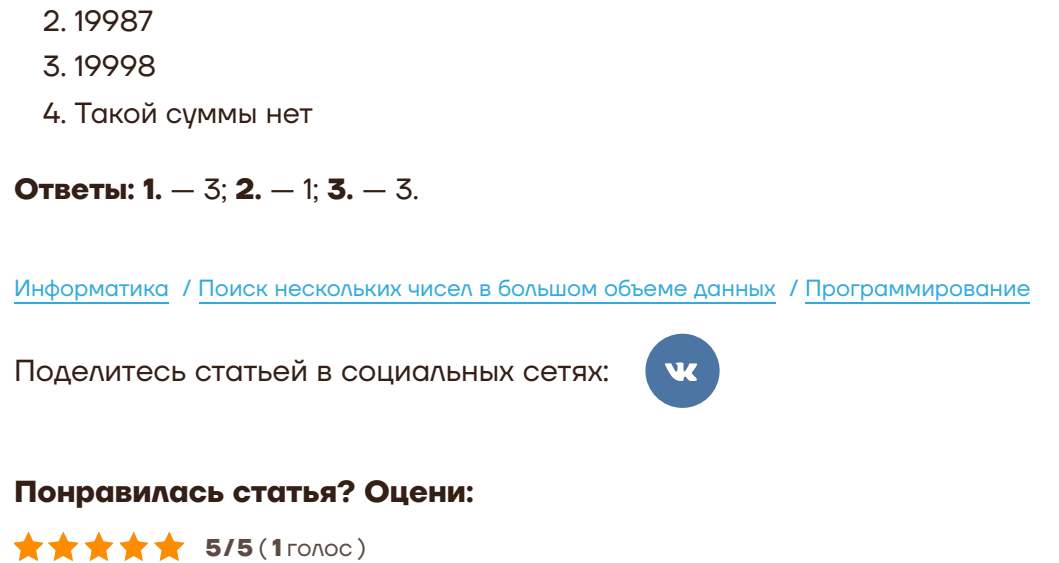
Информатика / Поиск нескольких чисел в большом объеме данных / Программирование

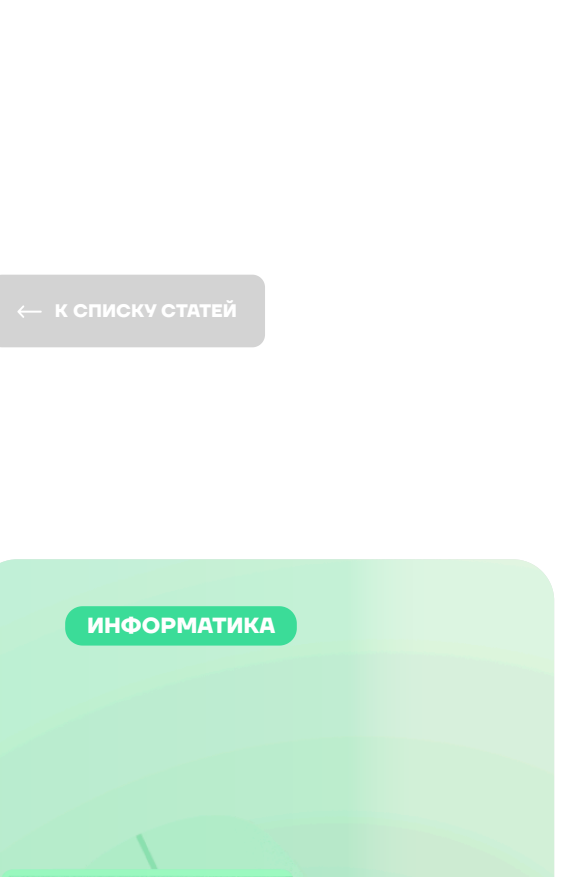
Поделитесь статьей в социальных сетях:

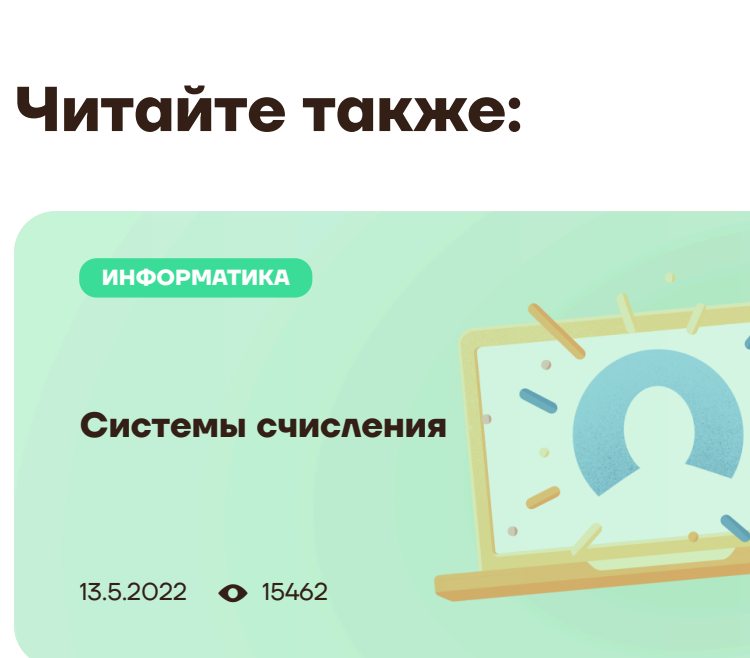
Понравилась статья? Оцени:
★★★★★ 5 / 5 (1 голос)

← К СПИСКУ СТАТЕЙ

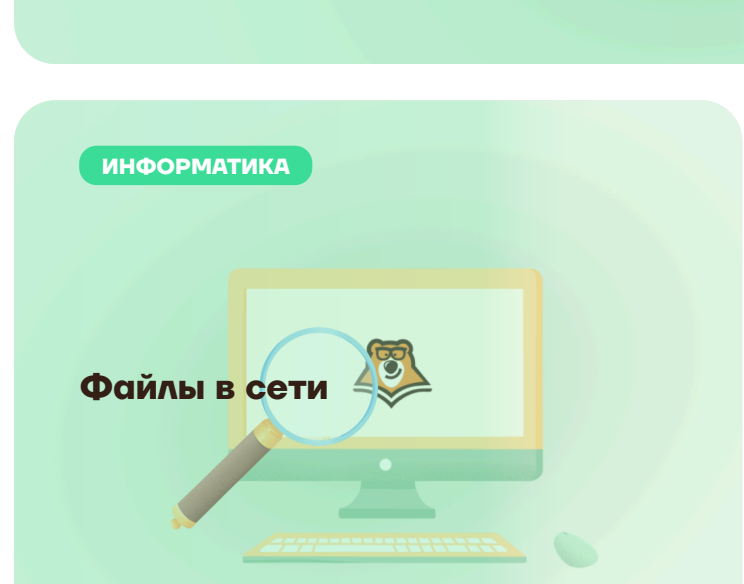
Читайте также:


ИНФОРМАТИКА
Системы счисления
13.5.2022 15462


ИНФОРМАТИКА
Практика анализа алгоритмов
3.5.2022 9423


ИНФОРМАТИКА
Файлы в сети
7.11.2023 4164

ИНФОРМАТИКА
Практика оптимального программирования. Поиск нескольких чисел в большом объеме данных
19.5.2022 5306


ИНФОРМАТИКА
Алгебра логики
14.5.2022 28585

ИНФОРМАТИКА
Основы программирования на языке Python. Часть 1
3.5.2022 33633

Подготовься к экзаменам на с Умскул!

Получи 80+ за ЕГЭ и 5 за ОГЭ

4 000+ стобалльников | 500 000+ учеников за 8 лет | 54 000+ получили 5 на ОГЭ

ТОЖЕ ТАК ХОЧУ

© УМСКУЛ, 2025